

Plan — génération procédurale de maps & missions

*Plan d'exploration : générer des cartes **et** les missions qui vont avec, en essayant plusieurs techniques, puis en gardant les meilleures via auto-évaluation. Ancré sur notre codebase (maillage Voronoï « île », terrain/relief/murets, pods, triggers) et inspiré des données XCOM 2.*

0. Pourquoi (et le cadre DIRECTION)

La DIRECTION choisit **A6 : procédural borné + golden path** — variété sans perdre le contrôle. Donc : pas du « tout aléatoire », mais des cartes **curées**. On a déjà la brique géométrique (`genMesh` → maillage irrégulier en île) ; il manque de **peupler** ce maillage (couvert, relief, pods, objectifs) **et** d'émettre la **mission** correspondante (slots de déploiement, ennemis, conditions, dialogues).

1. Ce que XCOM 2 nous apprend (données)

Système **Plot + Parcel + PCP** :

- **Plot** : la trame **faite main** (réseau de routes/chemins avec des `sockets`), commune à plein de missions → garantit la lisibilité.
- **Parcels** : modules (petit / moyen / grand, mesurés en **tuiles**) tirés au sort et encastrés dans les `sockets`.
- **PCP** (*Plot Cover Parcels*) : remplissage entre les parcellés, ~8 à 16 **tuiles** de long, largeur de rue.
- **Tuile = 96 unités Unreal** (empreinte d'une case).
- **Asset swapping** : tout est créé en « tempéré », puis les assets se substituent par biome (aride/tundra).
- **Philosophie (dir. artistique)** : « **Random's not fun** » — le pur procédural d'Enemy Unknown était laid ; le plot/parcel **mélange main + procédural** pour la rejouabilité sans le chaos.

Pods (groupes ennemis) :

- **2–3 ennemis par pod** (selon difficulté / force level ; parfois 1, jusqu'à plus avec mods).
- **Difficulté d'une mission ≈ nombre de pods** : *facile* 2–4, *moyen* 5–6, *difficile* 7–8.
- Un pod **patrouille** puis **s'active à la révélation** (mouvement libre / dispersion à l'activation) — mécanique qu'on a déjà (`pod` , `asleep` , `podAlerted` , réveil à vue).

Confiance. Système plot/parcel & philosophie : **source dev** (talk GDC 2018 « Plot and Parcel », B. Hess). Tuile 96 u, tailles PCP, tailles pods & nb de pods : **wikis/guides communautaires** (dimensions exactes des parcellés non publiées). À calibrer chez nous de toute façon (notre maillage n'est pas une grille).

2. Notre contexte (ce qui transpose, ce qui diffère)

- **Diffère** : notre maillage est un **Voronoï irrégulier** (cases de taille variable), pas une grille carrée. Les « tuiles » XCOM ne s'appliquent pas tel quel — on raisonne en **nombre de cellules**, **adjacence**, et **distance en sauts (hops)**.

- **Transpose directement** : la **densité de couvert**, le **dimensionnement/placement des pods**, le **plot fait-main** (= notre golden path / régions clés), la séparation **déploiement** ↔ **objectif** ↔ **pods**.
 - **On a déjà** : `genMesh` (forme), le générateur de **profil de terrain auto**, les **murets** (couvert d'arête), le **rocher** (bloquant), le **relief** (élévation → bonus/coût), les **pods + sommeil**, les **triggers** (dialogues/événements/zones), le format **mission** complet, et `tools/balance.js` (**simulateur IA vs IA**).
-

3. Les couches à générer

Une carte jouable = 5 couches. On peut générer chacune par plusieurs techniques (§4) :

1. **Forme** — le maillage île (déjà : `genMesh`, taille/aspect/densité).
 2. **Relief** — champs d'élévation (collines, crêtes, cuvettes).
 3. **Couvert & terrain** — rocher (bloquant/vue), terrain accidenté (coût), **murets** sur arêtes (couvert partiel) → vise une **densité cible**.
 4. **Pods & déploiement** — slots de déploiement joueur, pods ennemis (taille, classe, sommeil), éventuels PNJ/otages.
 5. **Objectif & scénarisation** — type de mission, zones de déclenchement, dialogues, conditions (`protect` / `loot` /win/lose), renforts.
-

4. Techniques à essayer (le cœur de l'exploration)

On veut **comparer** plusieurs générateurs par couche, derrière une interface commune (`generate(seed, params)` → `mission JSON`).

A. Scatter par densité (XCOM-like) — baseline rapide. Tirer du couvert/rocher/muret jusqu'à atteindre un **ratio cible** (ex. ~20–30 % des cellules en couvert, N murets / M arêtes). Simple, ajustable, bon point de départ.

B. Stamping de prefabs / « parcels » — la leçon XCOM (curé). Une bibliothèque de **chunks faits main** (un gué, une ruine-chokepoint, une clairière, un campement) stockés comme petits motifs (terrain+murets+élévation relatifs). On en **estampille** quelques-uns sur des régions du maillage, le reste rempli par scatter. → variété **maîtrisée**.

C. Bruit (Perlin/value noise). Champs continus pour l'élévation et les zones accidentées (relief crédible, gradients). Prolonge le « profil de terrain auto » existant.

D. Automates cellulaires. Grossir/lisser des amas de couvert → zones denses **vs** lanes ouvertes organiques (couloirs naturels).

E. Partition (BSP / régions). Découper l'île en sous-zones (salles/secteurs) reliées par des passages → cartes « intérieures » ou fronts. Mappé sur les cellules via leurs centroïdes.

F. WFC (Wave Function Collapse) — avancé. Cohérence locale des types de terrain par contraintes d'adjacence (évite le bruit incohérent). À tester si A–E ne suffisent pas à la lisibilité.

G. Placement par graphe (objectifs/pods) — transversal. Sur l'adjacence du maillage : choisir **déploiement** (un bord de l'île), **objectif(s)** (loin du déploiement), **pods** espacés le long du/des chemins, près du couvert, **hors de vue** du déploiement au départ. Contraintes par **hops** + **ligne de vue** (`los`, `hops` existent déjà).

5. Dimensionnement & placement des pods (règles dérivées de XCOM)

- **Taille de pod** : 2–3 unités (paramétrable ; varie avec la « difficulté » de la région).
 - **Nombre de pods par difficulté** : facile 2–4, moyen 5–6, difficile 7–8 (à l'échelle de nos cartes, sans doute revoir à la baisse — calibrer §7).
 - **Placement** : pods **hors de portée de vue** du déploiement au tour 1 ; **espacés** entre eux ($\geq K$ hops) pour éviter les multi-activations ; un pod **gardien d'objectif** ; certains **endormis** (notre `asleep`) pour l'approche furtive.
 - **Renforts** (optionnel) : pod qui apparaît au tour N ou sur trigger — via un événement scripté.
 - **Densité de couvert** : assez pour que les **lanes ouvertes soient un choix**, pas un vide. Cible de départ ~25 % de cellules-couvert + murets sur ~1 arête sur 6 ; à régler par simulation.
-

6. Générer la MISSION, pas seulement la carte

La sortie doit être un **mission JSON** que le moteur charge déjà :

- **Slots de déploiement** (Phase A : cases joueur) — placés sur un bord sûr.
 - **Unités ennemies** : `cls`, `pod`, `asleep` selon les pods générés ; PNJ/otages si archétype « sauvetage ».
 - **Conditions** : `protect` / `loot` / objectif ; **archétypes** : *élimination*, *sauvetage* (otages = PNJ à protéger), *extraction* (atteindre une zone), *défense* (survivre N tours), *sabotage* (atteindre une cible).
 - **Triggers** : dialogue d'intro/ambiance, révélation de pod, renforts, ouverture/fermeture de zone — composés à partir de modèles.
 - **Métadonnées geoscape** : difficulté, archétype, biome → la région du geoscape demande « une mission *sauvetage*, *moyenne* » et le générateur la produit.
-

7. Auto-évaluation (la clé pour « garder les bonnes »)

On a `tools/balance.js` + `tools/engine.js` (**simulateur IA vs IA headless**). Boucle proposée :

1. **Générer** N cartes/missions (techniques variées, seeds variés).
2. **Simuler** chacune K fois (IA joueur vs IA ennemi).
3. **Mesurer** : taux de victoire, nb de tours, pertes, % de couvert utilisé, accessibilité (tout est-il atteignable ?), multi-activations de pods.
4. **Filtrer** : rejeter trop faciles / trop dures / dégénérées (objectif injoignable, pods empilés). Garder la fenêtre « tendue mais jouable ».
5. **Comparer les techniques** sur ces métriques → on **sait** lesquelles marchent, on ne devine pas.

C'est l'angle le plus fort : un générateur **mesuré**, pas seulement « joli ». Ça matérialise le « procédural borné » de la DIRECTION.

8. Plan d'attaque (phases)

État — août 2026 : P1 à P5 sont faits. Le générateur produit des missions jouables seedées avec archétypes d'objectif, le calibrage par simulation est branché, et le geoscape peuple ses régions. Les statuts par phase sont indiqués ci-dessous ; ce qui reste est signalé en fin de section.

- **✓ P1 — Scatter + pods + objectif (baseline jouable).** Technique A (couvert par densité) + G (placement par graphe) + émission mission (archétype *élimination*). Bouton «  Générer mission » dans l'éditeur (seedé). *Livrable : une mission générée jouable de bout en bout.*
- **✓ P2 — Auto-évaluation.** Brancher `balance.js` : générer → simuler → filtrer ; tableau de métriques. *Livrable : on garde/rejette automatiquement.*
- **🟡 P3 — Prefabs/parcels (curé).** (*partiel : génération et profils de terrain en place ; bibliothèque de chunks faits main non constituée.*) Technique B + bibliothèque de chunks faits main. *Livrable : variété maîtrisée, lisible.*
- **✓ P4 — Relief & cohérence.** Techniques C/D (bruit, automates) sur l'élévation et les amas ; éventuellement F (WFC) si besoin. *Livrable : terrains crédibles.*
- **✓ P5 — Archétypes de mission.** Sauvetage / extraction / défense / sabotage + triggers générés ; intégration **geoscape** (la région demande un archétype+difficulté). *Livrable : le geoscape peuple ses régions tout seul.*

9. Décisions ouvertes (à trancher avec toi)

- **Échelle** : nos cartes (~quelques dizaines de cellules) → réduire les comptes de pods XCOM ? (probablement 1–4 pods).
- **Curé vs varié** : quel équilibre prefabs (B) vs scatter (A) ? (la DIRECTION penche curé.)
- **Persistance** : missions générées **figées** (sauvegardées comme JSON, donc rejouables/partageables) ou **régénérées** à la volée par seed ? (figer colle mieux à « l'échec fait avancer » et au golden path).
- **Déformation/thème** (à réorienter — *nouvel univers*) : le sens s'inverse. On part d'une **grille régulière** (la carte du Consortium) que l'organique **reconquiert**, jusqu'à **bloquer le passage** — la difficulté monte donc par *fermeture du terrain*, pas seulement par densité d'obstacles. Le moteur de maillage sait déjà produire les deux extrêmes (grille ↔ maillages fractals à tailles variables, avec trous, conformes et connexes par arêtes) ; reste à piloter la bascule par la progression de campagne.

Ce qui reste : la bibliothèque de **prefabs faits main** (P3), et surtout le branchement de la **génération sur le thème inversé** (grille → organique qui ferme le terrain).